

Complete SQL Interview Guide

Concepts, interviewer questions, strong answers, worked queries, query reasoning, and SDET database-validation scenarios

Purpose: This is an interview-preparation guide, not just a syntax reference. For each major concept it shows how an interviewer may ask the question, how to answer verbally, and how the query works step by step.

Recommended order: Fundamentals -> filtering -> aggregation -> joins -> subqueries/CTEs -> set operations -> indexes/transactions -> window functions -> practical interview problems -> SDET/API database validation.

Contents

1. SQL fundamentals and database terminology
2. Keys and constraints
3. NULL and comparison behavior
4. SELECT, WHERE, DISTINCT, ORDER BY, LIMIT/TOP
5. Aggregate functions, GROUP BY and HAVING
6. Joins
7. Subqueries, IN and EXISTS
8. CTEs
9. UNION and UNION ALL
10. CASE expressions
11. DELETE, TRUNCATE and DROP
12. Indexes and query performance
13. Normalization and denormalization
14. Transactions and ACID
15. Window functions
16. Ranking functions
17. Aggregate windows
18. LAG, LEAD and value functions
19. Window frames
20. Common SQL coding interview problems
21. SDET/API-to-database validation
22. Rapid interview cheat sheet

1. SQL Fundamentals

Interview question: What is SQL?

Strong answer: SQL stands for Structured Query Language. It is used to define, retrieve, insert, update, delete, and control data in relational databases. SQL is a language; products such as MySQL, PostgreSQL, Oracle Database, and SQL Server are database-management systems that implement SQL with some syntax differences.

Interview question: What is the difference between SQL and MySQL?

Answer: SQL is the language used to communicate with relational databases. MySQL is a relational database-management system that supports SQL.

Interview question: What is a table, row, and column?

Answer: A table stores related data. A row represents one record or entity instance, and a column represents one attribute of that record.

Example table used throughout this guide:

id	name	department_id	salary
1	Amit	10	80000
2	Ravi	10	60000
3	Priya	10	60000
4	Neha	20	70000
5	Rahul	20	50000

2. Keys and Constraints

Interview question: What is a primary key?

Answer: A primary key uniquely identifies each row. Its value must be unique and cannot be NULL. A primary key may contain one column or multiple columns.

Interview question: What is a foreign key?

Answer: A foreign key creates a relationship between tables by referencing a primary key or other eligible unique key in another table. It helps enforce referential integrity.

```
CREATE TABLE departments (  
  id INT PRIMARY KEY,  
  name VARCHAR(100) UNIQUE  
);  
  
CREATE TABLE employees (  
  id INT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  department_id INT,  
  salary DECIMAL(10,2) CHECK (salary >= 0),  
  FOREIGN KEY (department_id) REFERENCES departments(id)  
);
```

Interview question: Primary key vs UNIQUE?

Answer: Both enforce uniqueness, but a primary key is the table's principal row identifier and cannot be NULL. A table has one primary-key constraint, although it may be composite. Multiple UNIQUE constraints are allowed. Exact NULL behavior for UNIQUE constraints differs by database.

Common constraints: PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, and DEFAULT.

3. NULL

Interview question: What is NULL? Is it the same as 0 or an empty string?

Answer: NULL represents missing or unknown data. It is not the number 0 and is not an empty string.

Interview question: How do you check for NULL?

```
SELECT *
FROM employees
WHERE department_id IS NULL;

SELECT *
FROM employees
WHERE department_id IS NOT NULL;
```

Do not write `department_id = NULL`. SQL uses **IS NULL** because NULL represents an unknown value.

Interview question: What does COUNT(column) do with NULLs?

Answer: COUNT(*) counts rows. COUNT(column) counts non-NULL values in that column.

4. SELECT, WHERE, DISTINCT and ORDER BY

Interview question: Find employees earning more than 60,000.

```
SELECT *
FROM employees
WHERE salary > 60000;
```

Working: FROM identifies the table; WHERE filters rows; SELECT chooses the columns returned.

Interview question: Find unique department IDs.

```
SELECT DISTINCT department_id
FROM employees;
```

Interview question: Sort employees from highest to lowest salary.

```
SELECT name, salary
FROM employees
ORDER BY salary DESC;
```

Interview note: ORDER BY ASC is ascending and DESC is descending. Without ORDER BY, do not assume a guaranteed output order.

Interview question: Return only the first 5 records.

Answer: Syntax differs by DB. MySQL/PostgreSQL commonly use LIMIT; SQL Server commonly uses TOP or OFFSET/FETCH.

```
-- MySQL / PostgreSQL
SELECT * FROM employees
ORDER BY id
LIMIT 5;

-- SQL Server example
SELECT TOP 5 *
FROM employees
ORDER BY id;
```

5. Aggregate Functions, GROUP BY and HAVING

Important aggregates: COUNT(), SUM(), AVG(), MIN(), MAX().

Interview question: Find the average salary.

```
SELECT AVG(salary) AS average_salary
FROM employees;
```

Interview question: Find average salary for every department.

```
SELECT department_id, AVG(salary) AS average_salary
FROM employees
GROUP BY department_id;
```

Working: GROUP BY creates one group for each department_id. AVG is calculated independently for each group. The output has one row per group.

Interview question: What is the difference between WHERE and HAVING?

Strong answer: WHERE filters individual rows before grouping. HAVING filters grouped/aggregated results after GROUP BY.

```
SELECT department_id, COUNT(*) AS employee_count
FROM employees
WHERE salary >= 50000
GROUP BY department_id
HAVING COUNT(*) > 2;
```

Working: First WHERE removes rows with salary below 50,000. Then remaining rows are grouped by department. COUNT is calculated. Finally HAVING keeps only groups whose count is greater than 2.

6. Joins

Assume departments(id, department_name) and employees(..., department_id).

Interview question: What is an INNER JOIN?

Answer: INNER JOIN returns rows for which the join condition matches in both tables.

```
SELECT e.name, d.department_name
FROM employees e
INNER JOIN departments d
ON e.department_id = d.id;
```

Interview question: INNER JOIN vs LEFT JOIN?

Answer: INNER JOIN returns only matched rows. LEFT JOIN returns every row from the left table and matching data from the right; when there is no match, right-side columns are NULL.

```
SELECT e.name, d.department_name
FROM employees e
LEFT JOIN departments d
ON e.department_id = d.id;
```

Interview question: Find employees that have no matching department.

```
SELECT e.*
FROM employees e
LEFT JOIN departments d
ON e.department_id = d.id
WHERE d.id IS NULL;
```

Working: LEFT JOIN preserves every employee. Missing department matches produce NULL on the department side. WHERE d.id IS NULL keeps only those unmatched rows.

Interview question: What is a SELF JOIN?

Answer: A self join joins a table to itself, often for hierarchical relationships such as employee -> manager.

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m
ON e.manager_id = m.id;
```

Interview question: What is a CROSS JOIN?

Answer: CROSS JOIN returns the Cartesian product: every row from the first table combined with every row from the second. If A has 3 rows and B has 4, the result has 12 rows.

7. Subqueries, IN and EXISTS

Interview question: Find employees earning more than the company average.

```
SELECT *
FROM employees
WHERE salary > (
  SELECT AVG(salary)
  FROM employees
);
```

Working: The inner query calculates one average salary. The outer query compares each employee salary against that value.

Interview question: What is a correlated subquery?

Answer: A correlated subquery references a column from the outer query. Conceptually, it is evaluated in relation to each outer row, although the optimizer may transform execution.

```
SELECT e.*
FROM employees e
WHERE salary > (
  SELECT AVG(e2.salary)
  FROM employees e2
  WHERE e2.department_id = e.department_id
);
```

This finds employees earning more than their own department average.

Interview question: IN vs EXISTS?

Answer: IN tests whether a value belongs to a set returned by a subquery. EXISTS tests whether the subquery produces at least one row. EXISTS is especially natural for existence/correlated checks. Performance depends on the database and execution plan, so avoid claiming that one is always faster.

```
SELECT *
FROM employees e
WHERE EXISTS (
  SELECT 1
  FROM departments d
  WHERE d.id = e.department_id
);
```

8. CTEs - Common Table Expressions

Interview question: What is a CTE?

Answer: A CTE is a named temporary result set defined with WITH and used by the following statement. It can make complex SQL easier to read and can also support recursion in databases that implement recursive CTEs.

```
WITH department_average AS (
  SELECT department_id, AVG(salary) AS avg_salary
  FROM employees
  GROUP BY department_id
)
SELECT e.name, e.salary, d.avg_salary
FROM employees e
JOIN department_average d
ON e.department_id = d.department_id
WHERE e.salary > d.avg_salary;
```

Working: The CTE first creates department averages. The main query joins each employee to the appropriate average and keeps employees whose salary is above it.

Interview question: CTE vs subquery?

Answer: Both can express many of the same queries. CTEs are often clearer when a logical result needs a meaningful name or when a query has multiple stages. Do not assume a CTE is automatically faster; optimization

depends on the DB.

9. UNION and UNION ALL

Interview question: What is the difference between UNION and UNION ALL?

Answer: UNION combines compatible result sets and removes duplicate rows. UNION ALL combines them without duplicate removal, so it is usually cheaper when duplicate elimination is unnecessary.

```
SELECT email FROM customers
UNION
SELECT email FROM leads;

SELECT email FROM customers
UNION ALL
SELECT email FROM leads;
```

The SELECT statements must return compatible column counts/types.

10. CASE Expressions

Interview question: Categorize employees as High, Medium, or Low salary.

```
SELECT
name,
salary,
CASE
WHEN salary >= 80000 THEN 'High'
WHEN salary >= 60000 THEN 'Medium'
ELSE 'Low'
END AS salary_band
FROM employees;
```

Working: CASE evaluates conditions from top to bottom and returns the value associated with the first matching condition.

11. DELETE, TRUNCATE and DROP

Interview question: Explain DELETE vs TRUNCATE vs DROP.

Command	What it does	WHERE?	Structure remains?
DELETE	Removes selected/all rows	Yes	Yes
TRUNCATE	Removes all rows from a table	No	Yes
DROP	Removes the database object itself	No	No

```
DELETE FROM employees WHERE id = 10;
TRUNCATE TABLE employees;
DROP TABLE employees;
```

Interview caution: Transactional behavior, identity reset behavior, logging details, and foreign-key restrictions vary by database. State the conceptual difference first and mention DB-specific behavior if asked.

12. Indexes and Query Performance

Interview question: What is an index?

Strong answer: An index is an auxiliary data structure that helps the database locate rows more efficiently for suitable queries, reducing the need for full table scans. The trade-off is additional storage and extra work when rows are inserted, updated, or deleted.

```
CREATE INDEX idx_users_email
ON users(email);
```

Interview question: Should we index every column?

Answer: No. Indexes have maintenance and storage cost. Choose indexes based on query patterns, selectivity, joins, filtering, sorting, and actual execution plans.

Interview question: What is a composite index?

Answer: An index containing multiple columns, for example (department_id, salary). Column order matters for how efficiently different query patterns can use the index.

Performance interview tip: Mention execution plans (EXPLAIN / EXPLAIN ANALYZE or DB equivalent), selective predicates, avoiding unnecessary columns, appropriate indexes, and measuring rather than guessing.

13. Normalization and Denormalization

Interview question: What is normalization?

Answer: Normalization organizes relational data to reduce unnecessary duplication and update anomalies.

1NF: values are atomic and repeating groups are removed.

2NF: in addition to 1NF, non-key attributes depend on the whole candidate key rather than only part of a composite key.

3NF: in addition to 2NF, non-key attributes should not depend transitively on another non-key attribute.

Interview question: What is denormalization?

Answer: Deliberately introducing some redundancy to improve read performance or simplify access patterns, while accepting extra consistency/maintenance cost.

14. Transactions and ACID

Interview question: What is a transaction?

Answer: A transaction is a logical unit of work containing one or more database operations that should be handled together.

```
BEGIN;  
  
UPDATE accounts  
SET balance = balance - 100  
WHERE id = 1;  
  
UPDATE accounts  
SET balance = balance + 100  
WHERE id = 2;  
  
COMMIT;
```

Interview question: Explain ACID.

Atomicity: the transaction succeeds as a unit or its changes are rolled back.

Consistency: a successful transaction preserves defined database rules/constraints.

Isolation: concurrent transactions are controlled so their interactions obey the configured isolation semantics.

Durability: committed changes survive failures according to the database's durability guarantees.

Interview question: How would an SDET test rollback?

Answer: Create a scenario where a multi-step transaction fails after an earlier step, then verify that partial changes were not committed. Validate both the API/error response and the final database state.

15. Window Functions - The Core Mental Model

Interview question: What is a window function?

Strong answer: A window function performs a calculation across a set of rows related to the current row while preserving the individual rows. Unlike GROUP BY, it does not collapse the result to one row per group.

Use this summary to remember the families:

```
OVER()  
|  
+-----+-----+  
| | |  
Ranking Aggregate Navigation  
| | |  
ROW_NUMBER SUM LAG  
RANK AVG LEAD  
DENSE_RANK COUNT FIRST_VALUE  
MIN LAST_VALUE  
MAX NTH_VALUE
```

The basic pattern is:

```
FUNCTION() OVER (  
PARTITION BY ...  
ORDER BY ...  
)
```

Read it in English: Apply FUNCTION over a defined window. PARTITION BY splits rows into independent windows. ORDER BY defines the calculation order within each window.

Interview question: What does PARTITION BY do?

Answer: It divides the result into independent groups for the window calculation. The function restarts for each partition.

```
RANK() OVER (  
PARTITION BY department_id  
ORDER BY salary DESC  
)
```

Read this as: "For each department, order employees from highest to lowest salary and assign ranks."

Interview question: PARTITION BY vs GROUP BY?

Answer: GROUP BY collapses multiple rows into one row per group. PARTITION BY defines groups for a window calculation but keeps the original rows.

16. ROW_NUMBER, RANK and DENSE_RANK

Salary	ROW_NUMBER	RANK	DENSE_RANK
100000	1	1	1
90000	2	2	2
90000	3	2	2
80000	4	4	3
70000	5	5	4
70000	6	5	4
60000	7	7	5

ROW_NUMBER(): every row receives a unique sequential number. Ties do not share numbers.

RANK(): equal ordering values share a rank, and the next rank has a gap. Pattern: 1, 2, 2, 4.

DENSE_RANK(): equal values share a rank, but no ranks are skipped. Pattern: 1, 2, 2, 3.

Interview question: Which function should I use for the second-highest distinct salary?

Answer: DENSE_RANK is a natural choice because equal salaries receive the same rank and the next distinct salary gets the next consecutive rank.

```
SELECT salary
FROM (
  SELECT
    salary,
    DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM employees
) x
WHERE rnk = 2;
```

Working: First rank distinct salary levels logically through DENSE_RANK. Highest salary receives 1. The next distinct salary receives 2 even if multiple people share rank 1. The outer query keeps rank 2.

Interview question: Why do we use an outer query/CTE to filter rank?

Answer: In standard query processing, WHERE filters before window-function results are available. Therefore we calculate the window value in a subquery/CTE and filter it outside. Some databases provide QUALIFY, but it is not universal.

17. Aggregate Window Functions

Interview question: Show each employee plus their department average.

```
SELECT
  name,
  department_id,
  salary,
  AVG(salary) OVER (
    PARTITION BY department_id
  ) AS department_avg
FROM employees;
```

Working: PARTITION BY forms a salary window for each department. AVG is calculated for each window. The same department average is attached to every employee in that department.

Interview question: Show each employee plus department employee count.

```
SELECT
  name,
  department_id,
  COUNT(*) OVER (
    PARTITION BY department_id
  ) AS department_count
FROM employees;
```

Interview question: Show the maximum department salary on every employee row.

```
SELECT
  name,
  salary,
  MAX(salary) OVER (
    PARTITION BY department_id
  ) AS department_max
FROM employees;
```

18. LAG, LEAD and Value Functions

Interview question: What is LAG()?

Answer: LAG retrieves a value from a preceding row relative to the current row according to the window ordering.

```
SELECT
  sale_date,
  amount,
  LAG(amount) OVER (ORDER BY sale_date) AS previous_amount
FROM sales;
```

Interview question: What is LEAD()?

Answer: LEAD retrieves a value from a following row.

```
SELECT
  sale_date,
  amount,
  LEAD(amount) OVER (ORDER BY sale_date) AS next_amount
FROM sales;
```

Interview question: Calculate change from previous sale.

```
SELECT
  sale_date,
  amount,
  amount - LAG(amount) OVER (ORDER BY sale_date) AS change_from_previous
FROM sales;
```

FIRST_VALUE: returns the first value in the relevant ordered frame.

LAST_VALUE: returns the last value in the relevant frame; default-frame behavior can surprise candidates.

NTH_VALUE: returns the nth value in the relevant frame where supported.

19. Window Frames and Running Totals

Interview question: How do you calculate a running total?

```
SELECT
  id,
  amount,
  SUM(amount) OVER (
    ORDER BY id
    ROWS BETWEEN UNBOUNDED PRECEDING
    AND CURRENT ROW
  ) AS running_total
FROM sales;
```

Working: For each row, the frame begins at the first row in the ordered window and ends at the current row. SUM therefore accumulates values as the result moves forward.

Important: ROWS and RANGE have different semantics, especially when ORDER BY values tie. Exact defaults and support can vary by DB, so for interviews state the frame explicitly when correctness matters.

20. Common SQL Coding Interview Problems

Interview question: 1. Find duplicate emails.

```
SELECT email, COUNT(*) AS occurrences
FROM users
GROUP BY email
HAVING COUNT(*) > 1;
```

Reasoning: GROUP BY creates one group per email. COUNT counts rows in each group. HAVING keeps groups appearing more than once.

Interview question: 2. Find the highest salary.

```
SELECT MAX(salary) AS highest_salary
FROM employees;
```

Interview question: 3. Find the second-highest distinct salary without a window function.

```
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (
  SELECT MAX(salary)
  FROM employees
);
```

Reasoning: Inner MAX finds the highest salary. WHERE removes that salary level. Outer MAX finds the largest remaining salary.

Interview question: 4. Find employees earning above the company average.

```
SELECT *
FROM employees
WHERE salary > (
  SELECT AVG(salary)
  FROM employees
);
```

Interview question: 5. Find the highest-paid employee(s) in every department.

```
WITH ranked AS (
  SELECT
    e.*,
    RANK() OVER (
      PARTITION BY department_id
      ORDER BY salary DESC
    ) AS rnk
  FROM employees e
)
SELECT *
FROM ranked
WHERE rnk = 1;
```

Why RANK? If multiple employees tie for the highest salary, all of them should be returned.

Interview question: 6. Return exactly one highest-paid employee per department.

```
WITH numbered AS (
  SELECT
    e.*,
    ROW_NUMBER() OVER (
      PARTITION BY department_id
      ORDER BY salary DESC, id ASC
    ) AS rn
  FROM employees e
)
SELECT *
FROM numbered
WHERE rn = 1;
```

Why id ASC? It provides a deterministic tie-breaker when salaries are equal.

Interview question: 7. Find departments having more than 5 employees.

```
SELECT department_id, COUNT(*) AS employee_count
FROM employees
GROUP BY department_id
HAVING COUNT(*) > 5;
```

Interview question: 8. Find departments with no employees.

```
SELECT d.*
FROM departments d
LEFT JOIN employees e
ON e.department_id = d.id
WHERE e.id IS NULL;
```

Interview question: 9. Find employees whose names start with A.

```
SELECT *
FROM employees
WHERE name LIKE 'A%';
```

Interview question: 10. Find the latest record for a user.

```
SELECT *
FROM user_events
WHERE user_id = 101
ORDER BY created_at DESC
LIMIT 1;
```

Use the equivalent TOP/OFFSET/FETCH syntax for databases that do not support LIMIT.

Interview question: 11. Find the Nth-highest distinct salary.

```
WITH ranked AS (
SELECT
salary,
DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
FROM employees
)
SELECT DISTINCT salary
FROM ranked
WHERE rnk = :n;
```

Interview question: 12. Find records in A that do not exist in B.

```
SELECT a.*
FROM table_a a
LEFT JOIN table_b b
ON b.id = a.id
WHERE b.id IS NULL;
```

Another common expression is NOT EXISTS.

```
SELECT a.*
FROM table_a a
WHERE NOT EXISTS (
SELECT 1
FROM table_b b
WHERE b.id = a.id
);
```

21. SDET and API-to-Database Interview Scenarios

Interview question: An API creates a user. How would you validate the database?

Strong answer: “I would call the create API, verify the expected status and response schema, capture the generated unique ID, query the database using that ID, and compare the fields that are contractually expected to persist. I would also validate generated/default fields when appropriate and make sure no duplicate record was created.”

```
SELECT id, name, email, status, created_at
FROM users
```

```
WHERE id = ?;
```

Automation flow: Request -> response validation -> extract ID -> DB query -> map DB row -> compare expected fields -> assertions -> cleanup.

Interview question: How would you validate an UPDATE API?

Answer: Read the existing state if needed, send the update, query the same unique ID, verify changed fields have the expected new values, verify fields that should remain unchanged, and validate audit/version fields if the application uses them.

Interview question: How would you validate a DELETE API?

Answer: First determine whether the system uses hard delete or soft delete. For hard delete, query by ID and expect no row. For soft delete, expect the row to remain with fields such as `deleted_at`/status updated.

Interview question: How would you compare 100 API records with database records?

Answer: Retrieve the corresponding DB dataset, normalize representations if required, map both datasets using a stable unique key, compare required fields by key, and separately validate missing/extra records. Do not rely on list order unless ordering is part of the API contract.

Interview question: How do you test database constraints?

Answer: Attempt invalid operations intentionally: duplicate a UNIQUE value, insert NULL into NOT NULL, use an invalid foreign key, violate CHECK constraints, and verify that the database/application rejects the operation without corrupting data.

Interview question: How do you test transactions?

Answer: Validate successful commit and forced-failure rollback. In a multi-step operation, create a failure after an earlier write and verify no partial state remains when atomicity is expected.

Interview question: How do you avoid flaky DB validations in automation?

Answer: Use unique test data and stable identifiers, understand asynchronous/eventual-consistency behavior, wait on a bounded condition rather than arbitrary sleeps when data is asynchronous, isolate test records, clean up safely, and avoid assertions on fields not guaranteed by the contract.

22. Rapid Interview Cheat Sheet

Topic	One-line interview answer
WHERE vs HAVING	WHERE filters rows before grouping; HAVING filters groups after aggregation.
INNER vs LEFT JOIN	INNER returns matches; LEFT preserves all left rows and fills missing right data with NULL.
COUNT(*) vs COUNT(col)	COUNT(*) counts rows; COUNT(col) counts non-NULL values.
GROUP BY vs window	GROUP BY collapses groups; windows preserve individual rows.
ROW_NUMBER	Unique sequence per row.
RANK	Ties share rank and create gaps.
DENSE_RANK	Ties share rank without gaps.
LAG / LEAD	Read previous / next row relative to the window order.
UNION / UNION ALL	UNION removes duplicates; UNION ALL keeps them.
Primary / foreign key	Primary identifies a row; foreign key enforces a relationship.
Index	Speeds suitable reads at the cost of storage and write maintenance.
Transaction	A logical unit of work that is committed or rolled back according to transaction semantics.

Topic	One-line interview answer
CTE	A named result set defined with WITH for the following statement.
EXISTS	Checks whether a subquery returns at least one row.

How to answer a coding question

1. Clarify ties, NULL behavior, and expected output if relevant.
2. State your approach before typing.
3. Write the query.
4. Walk through each clause.
5. Mention important edge cases.
6. If there are multiple solutions, explain why you chose yours.

Example verbal walkthrough

For “highest salary per department”: “I will partition employees by department, order salary descending, rank employees inside each department, and then filter rank 1. I am using RANK rather than ROW_NUMBER because if two employees tie for highest salary, I want both returned.”

Final topics to practice without notes

SELECT/WHERE/ORDER BY; NULL; keys and constraints; GROUP BY/HAVING; COUNT/SUM/AVG/MIN/MAX; INNER/LEFT/self joins; subqueries; IN/EXISTS; CTEs; UNION/UNION ALL; CASE; DELETE/TRUNCATE/DROP; indexes; normalization; transactions/ACID; ROW_NUMBER/RANK/DENSE_RANK; aggregate windows; LAG/LEAD; running totals; duplicates; Nth-highest salary; highest per department; records missing between tables; API-to-DB validation.